ShareWay: evidencias de pruebas en Postman

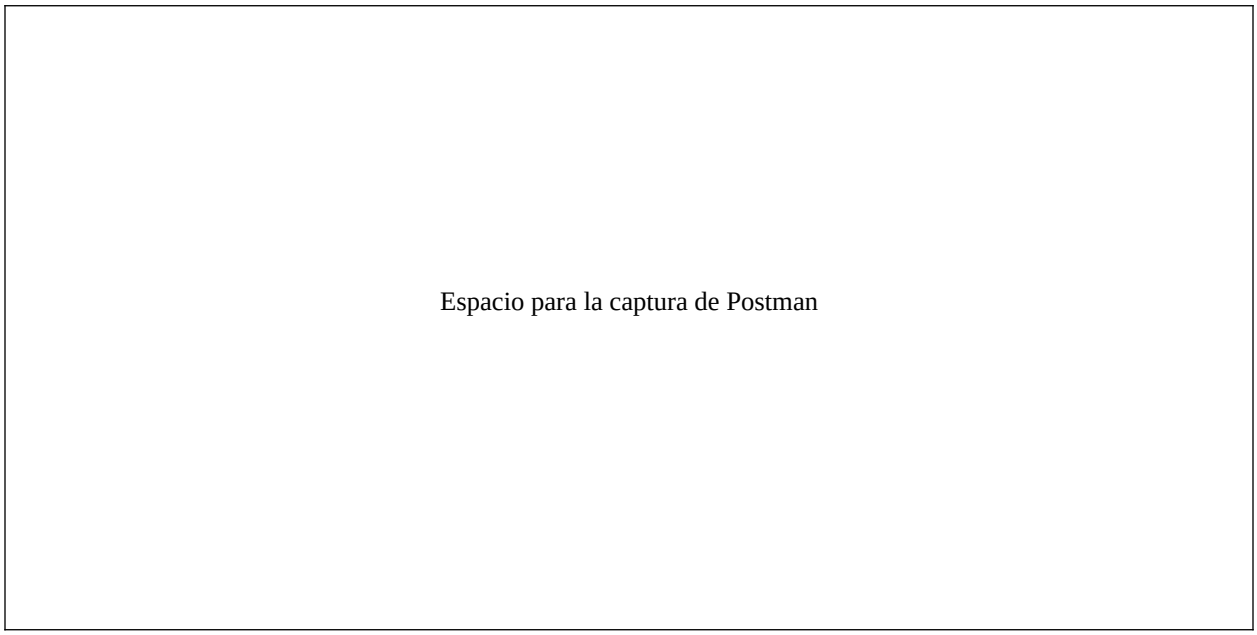
Trip Planning & Matching Service

Las pruebas se realizaron desde la aplicación Postman contra la API desplegada en Render. Las capturas compartidas muestran 16 casos y 28 aserciones aprobadas. Se comprobaron el estado del servicio, el registro de datos, el matching, la repetición sin duplicados y las respuestas ante entradas inválidas o ausencia de candidatos.

Figura 1. Salud del servicio

GET /actuator/health

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



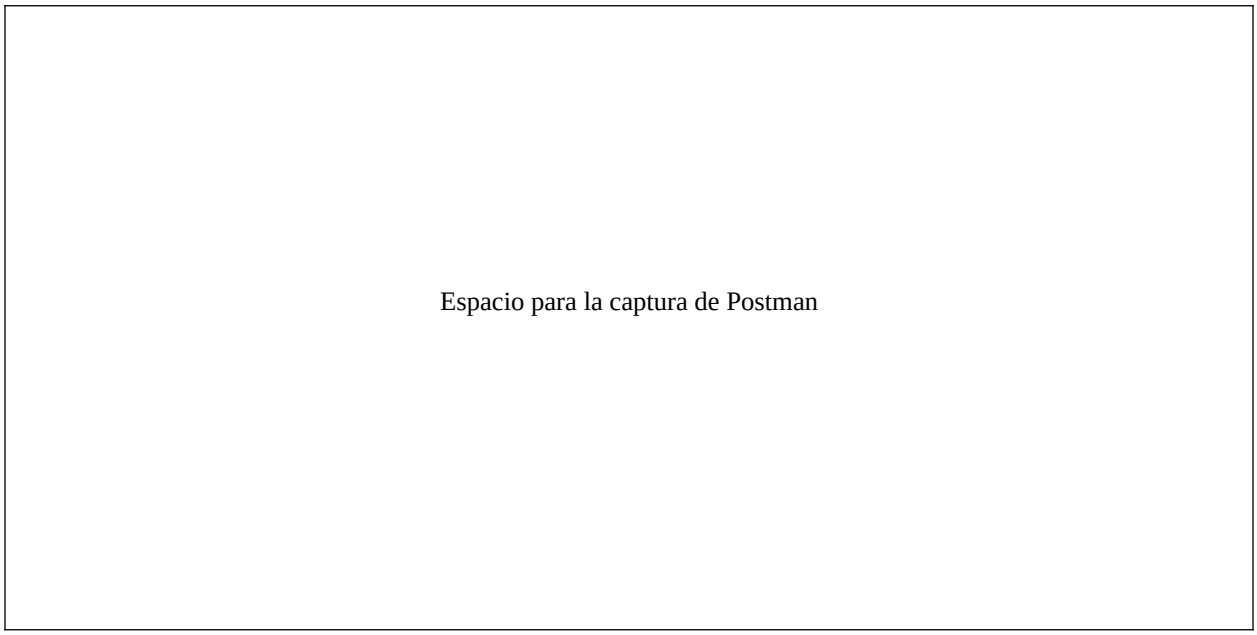
Se consultó el estado de la API desplegada en Render. La respuesta incluye status UP y los grupos liveness y readiness. Las dos aserciones visibles comprobaron el código HTTP y el estado del servicio.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 2. Registro de disponibilidad compatible

POST /api/v1/driver-availabilities

Respuesta observada: 201 Created. Pruebas aprobadas: 2/2.



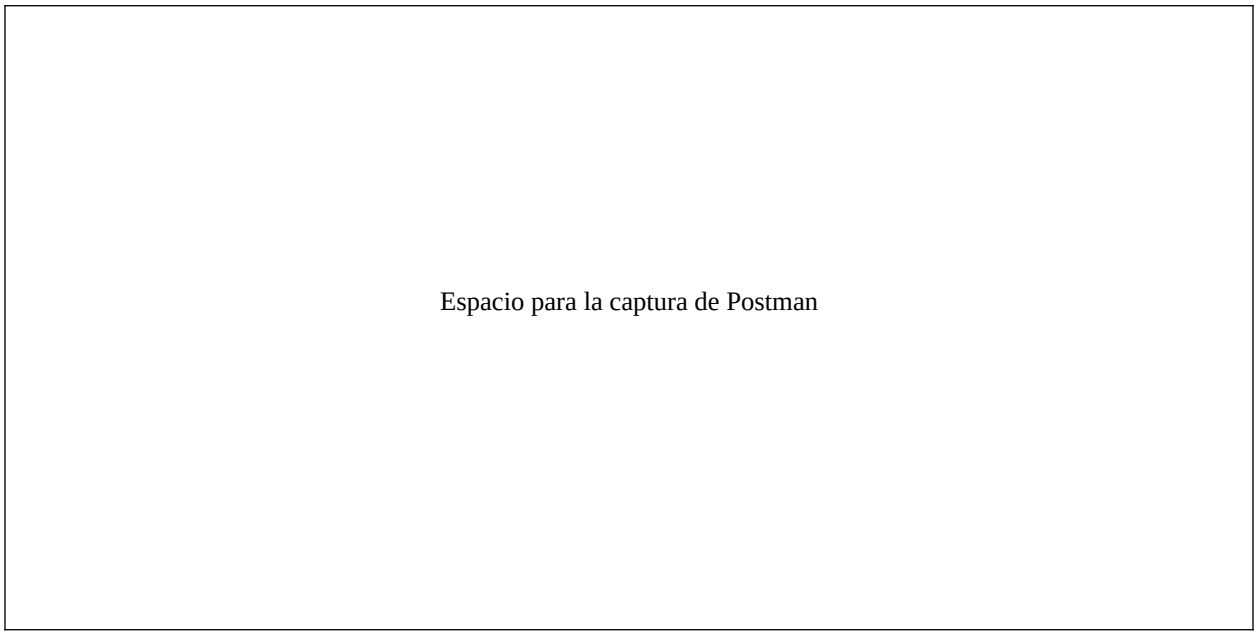
Se registró la disponibilidad del conductor 10 para el 10 de noviembre de 2026, entre las 07:00 y las 09:00, con tres asientos. El servidor devolvió el UUID del registro y los datos enviados. La variable zone se resolvió a una zona ficticia compartida por los casos de esta ejecución.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 3. Creación de solicitud de viaje

POST /api/v1/trip-requests

Respuesta observada: 201 Created. Pruebas aprobadas: 2/2.



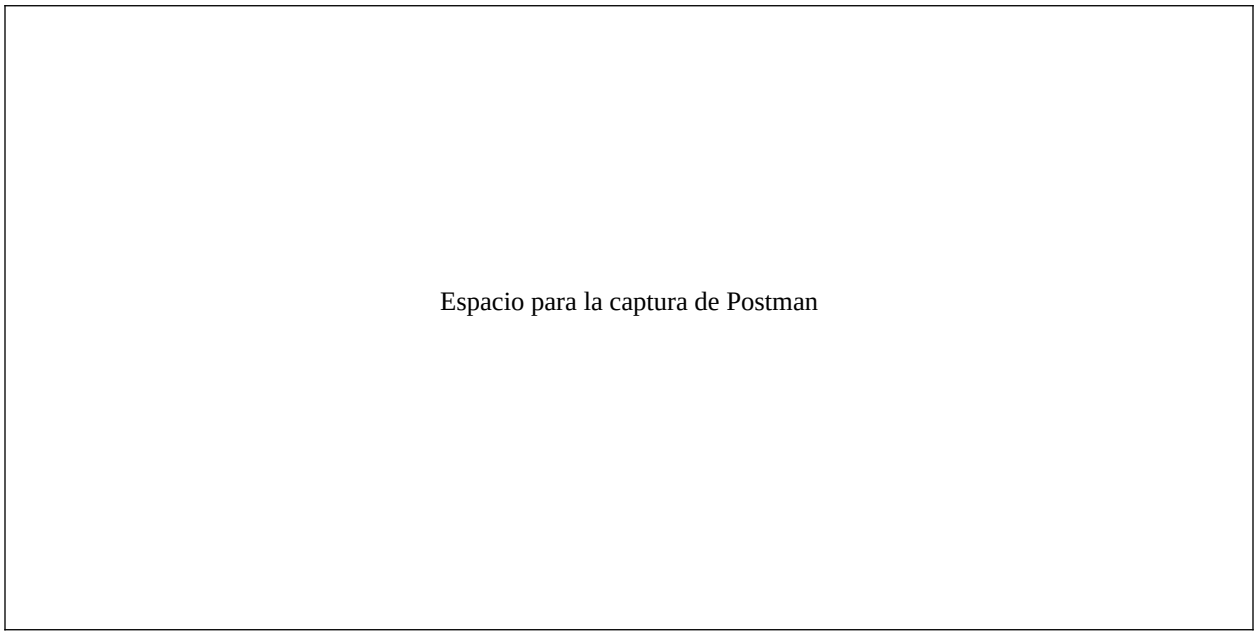
El pasajero 25 solicitó un asiento desde UPC Monterrico hacia Chorrillos para el 10 de noviembre de 2026, con salida entre las 07:30 y las 08:30. La solicitud utilizó la misma zona del conductor. La respuesta devolvió un UUID y el estado PENDING_MATCH.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 4. Consulta de solicitudes registradas

GET /api/v1/trip-requests

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



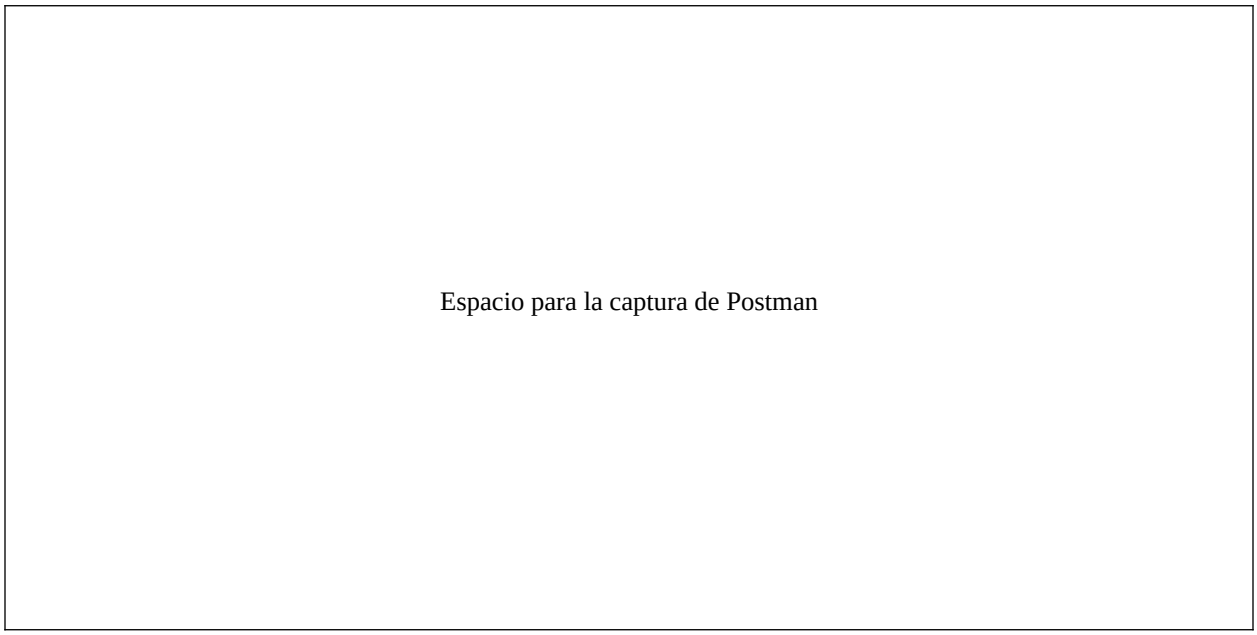
Se consultó la lista de solicitudes. La respuesta incluyó la solicitud creada en el caso anterior, junto con sus datos de viaje y estado PENDING_MATCH. La presencia de otros registros en la lista corresponde a datos que ya existían en la base del servicio.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 5. Generación de propuestas compatibles

POST /api/v1/trip-requests/{tripRequestId}/match-proposals

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



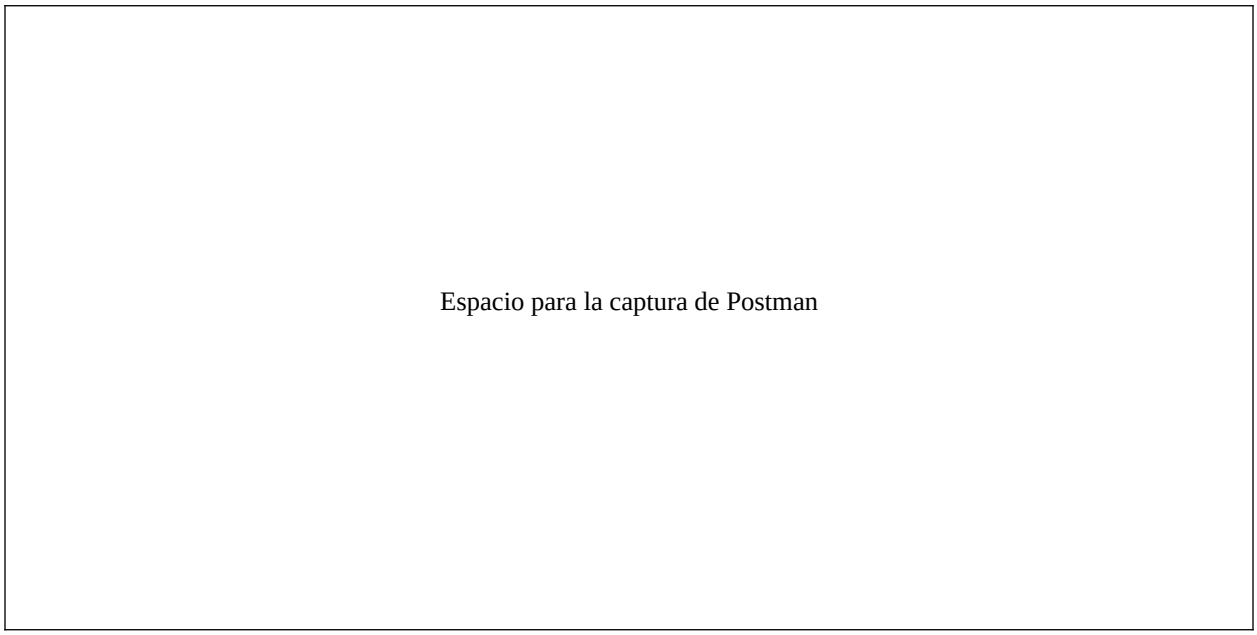
Se generó el matching para la solicitud registrada. La respuesta contiene una propuesta para el conductor 10, con estimatedFare 7.70, estimatedDurationMinutes 60 y status PROPOSED. El identificador de propuesta es 450a08d1-dad0-485f-adcb-591399449a8e.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 6. Repetición de generación sin duplicados

POST /api/v1/trip-requests/{tripRequestId}/match-proposals

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



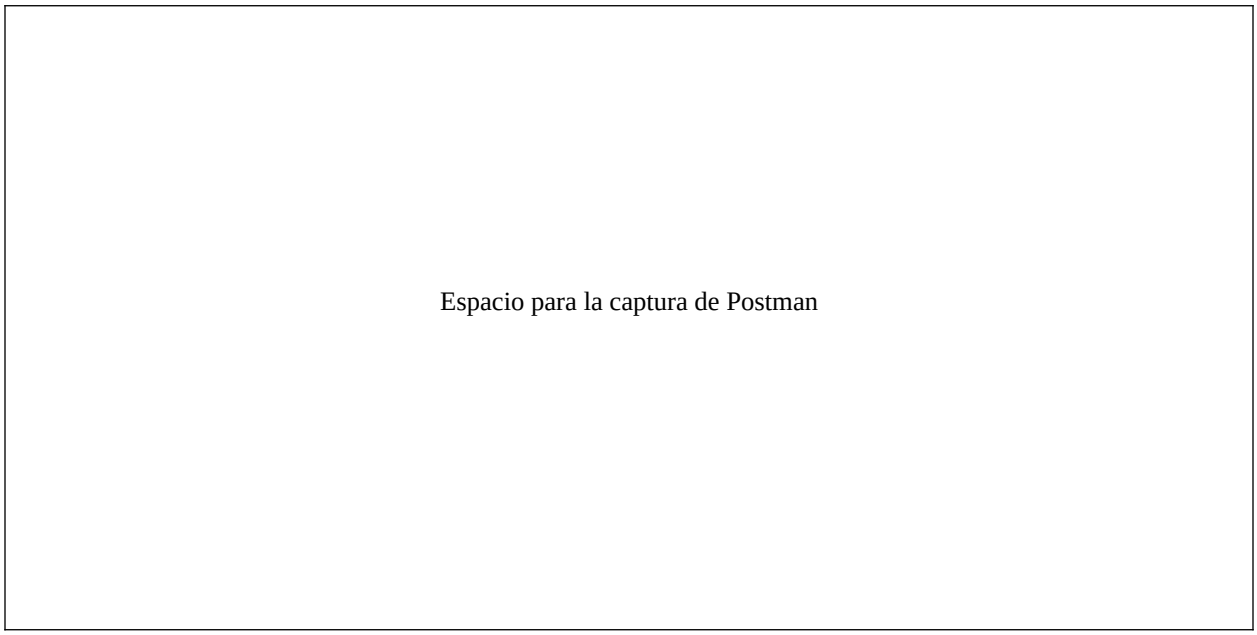
Se repitió la generación para la misma solicitud. El servidor devolvió una propuesta con el mismo identificador 450a08d1-dad0-485f-adcb-591399449a8e. La comparación confirma que esta repetición secuencial no creó otra propuesta.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 7. Consulta de propuestas guardadas

GET /api/v1/trip-requests/{tripRequestId}/match-proposals

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



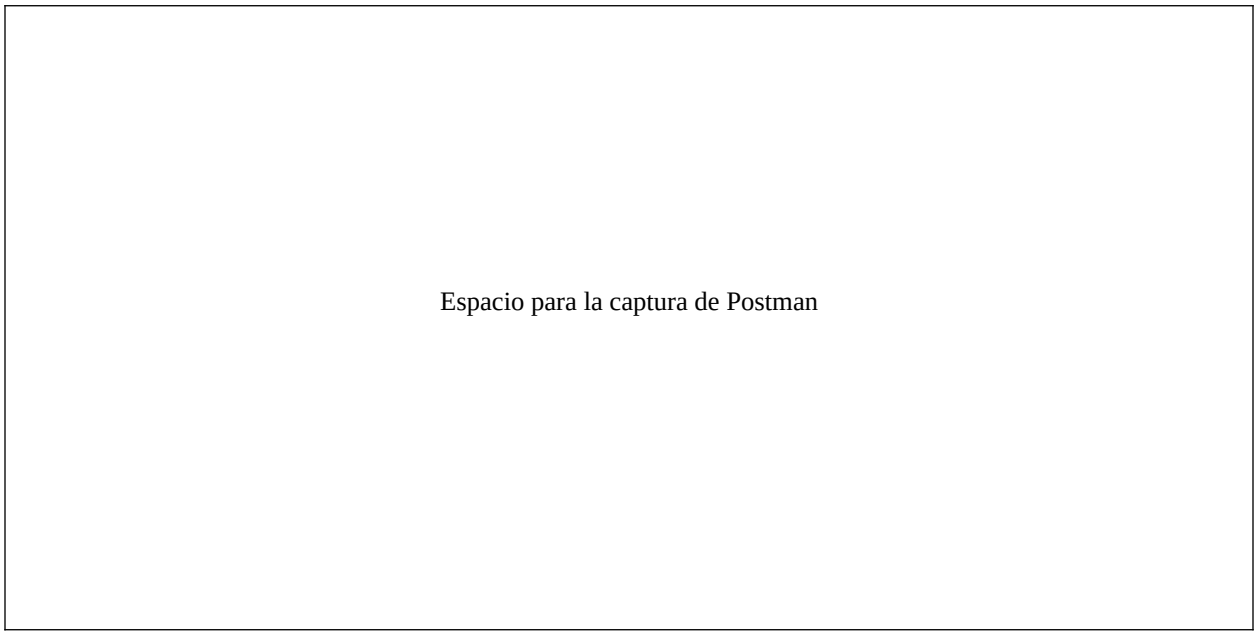
Se consultaron las propuestas almacenadas para la solicitud. La respuesta conserva el identificador y los datos de la propuesta generada: conductor 10, tarifa 7.70, duración 60 minutos y estado PROPOSED.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 8. Rechazo de solicitud vacía

POST /api/v1/trip-requests

Respuesta observada: 400 Bad Request. Pruebas aprobadas: 2/2.



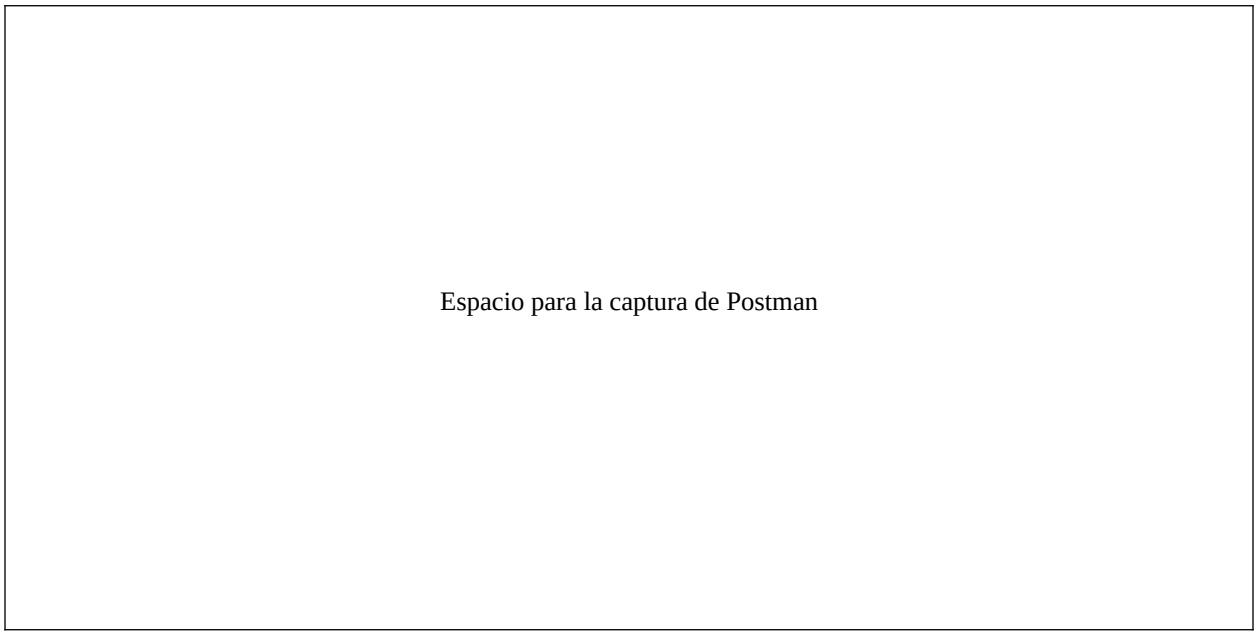
Se envió un objeto JSON vacío, sin los campos obligatorios de la solicitud. La API respondió con status 400, error Bad Request y la ruta de la operación. El resultado coincide con el rechazo esperado para una entrada incompleta.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 9. Rechazo de latitud fuera de rango

POST /api/v1/trip-requests

Respuesta observada: 400 Bad Request. Pruebas aprobadas: 2/2.



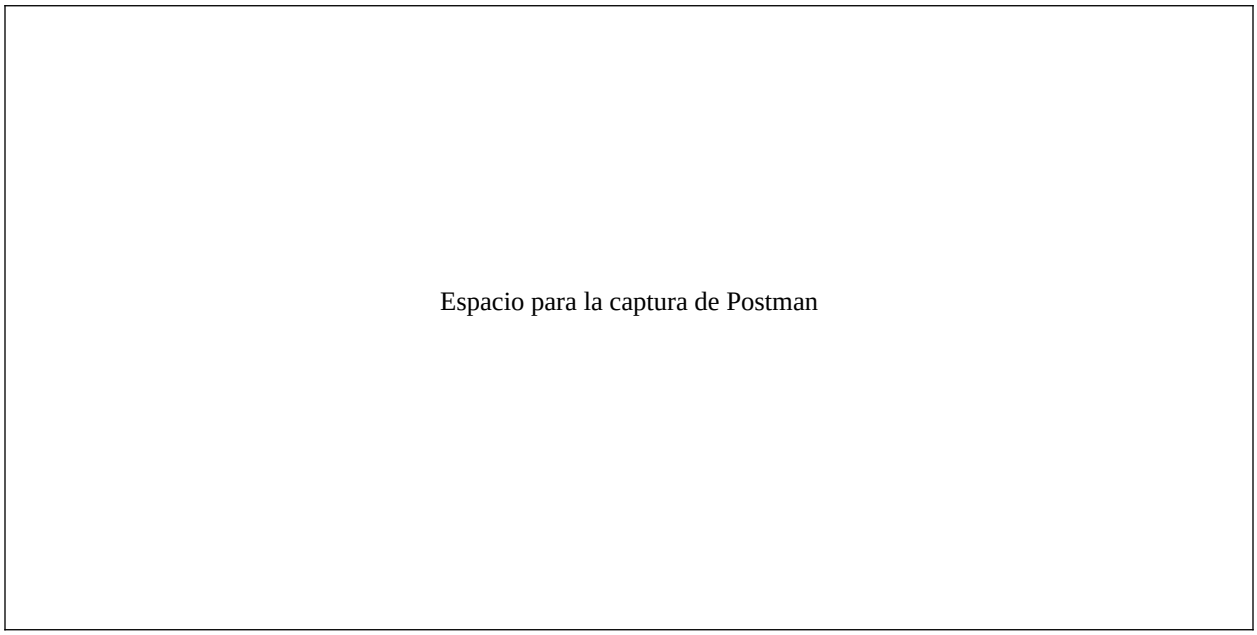
Se envió originLat con valor 100. El servidor respondió Validation failed y el mensaje originLat: must be less than or equal to 90.0. La respuesta identifica el campo y el límite incumplido.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 10. Rechazo de disponibilidad sin asientos

POST /api/v1/driver-availabilities

Respuesta observada: 400 Bad Request. Pruebas aprobadas: 1/1.



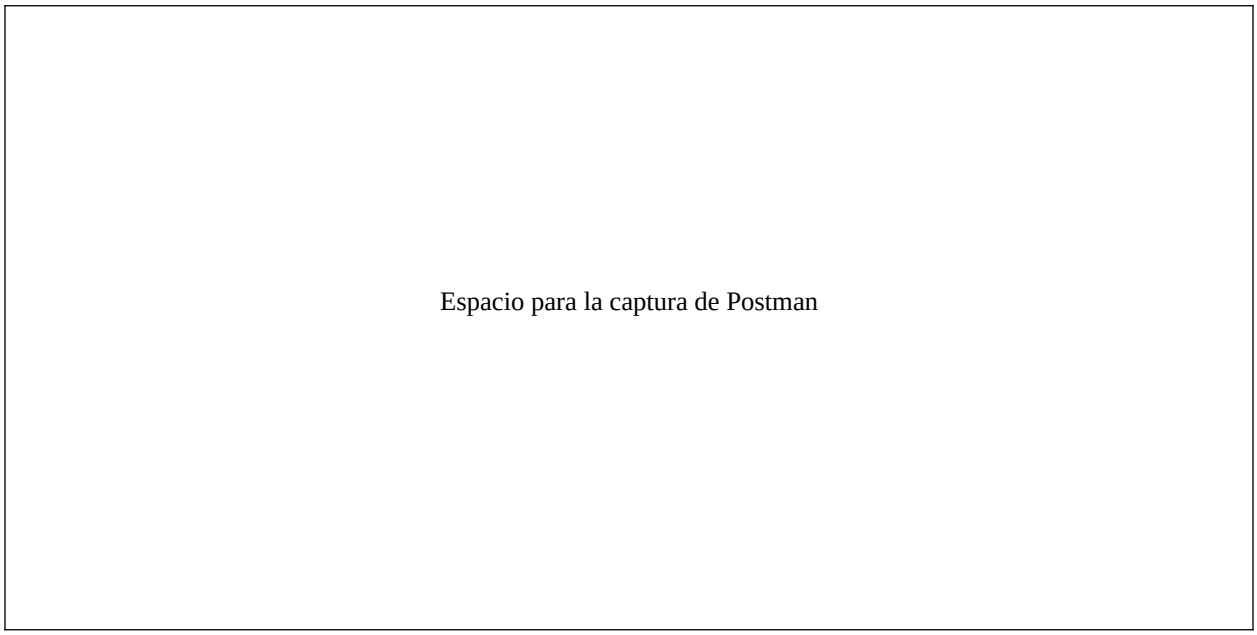
Se intentó registrar una disponibilidad con availableSeats igual a cero. La respuesta indica Validation failed y el mensaje availableSeats: must be greater than or equal to 1. El caso comprueba el mínimo de asientos permitido.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 11. Solicitud inexistente al generar matching

POST /api/v1/trip-requests/00000000-0000-0000-0000-000000000000/match-proposals

Respuesta observada: 404 Not Found. Pruebas aprobadas: 2/2.



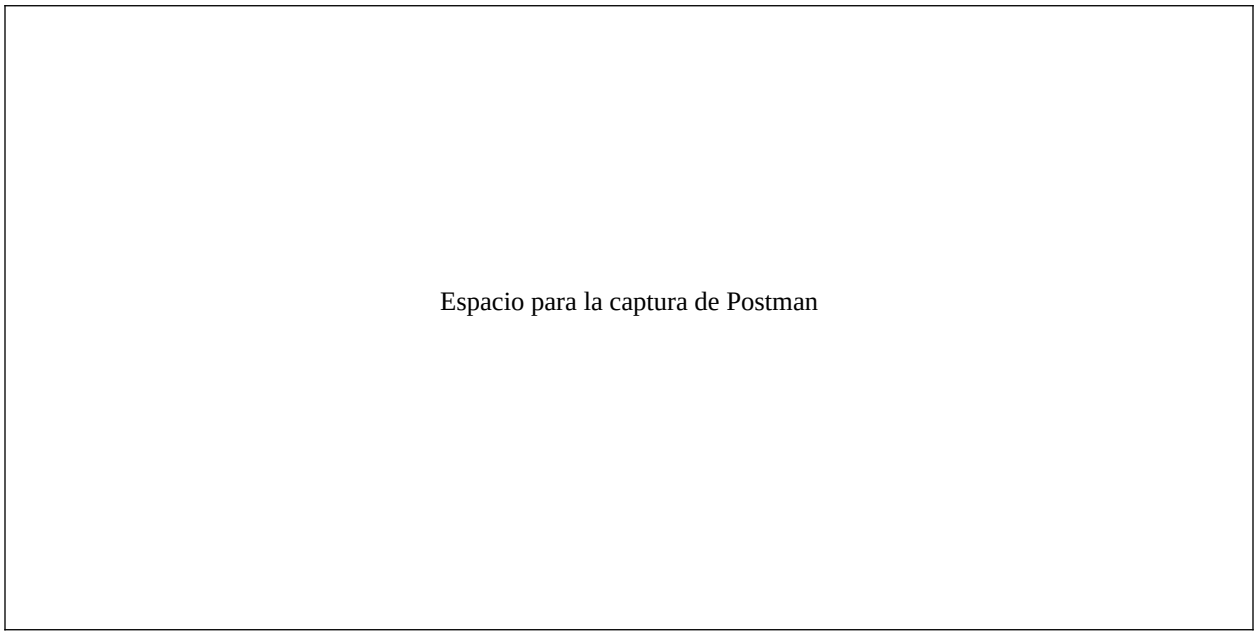
Se utilizó un UUID de formato válido que no corresponde a una solicitud registrada. La API devolvió status 404 y el mensaje Trip request not found, seguido del identificador enviado. Se distingue así un recurso inexistente de un identificador con formato inválido.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 12. Rechazo de UUID mal formado

POST /api/v1/trip-requests/no-es-un-uuid/match-proposals

Respuesta observada: 400 Bad Request. Pruebas aprobadas: 1/1.



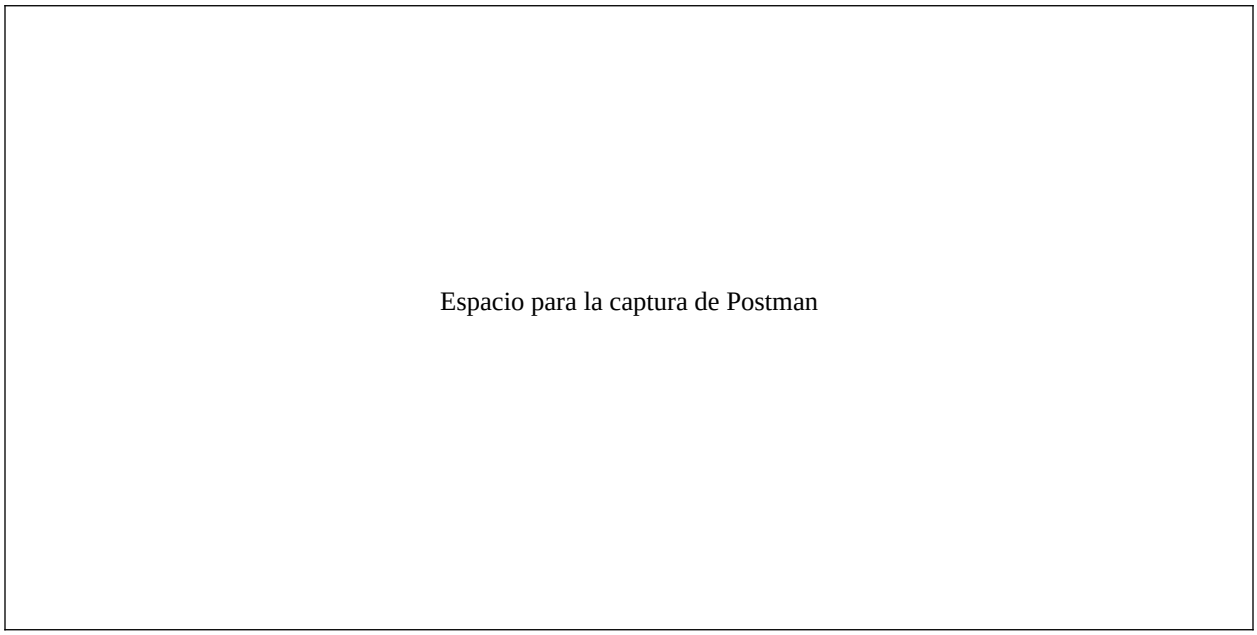
Se envió no-es-un-uuid como identificador de la solicitud. El servidor respondió status 400, error Bad Request y la ruta consultada. El resultado muestra el rechazo de un parámetro que no cumple el formato UUID.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 13. Solicitud en una zona sin conductores

POST /api/v1/trip-requests

Respuesta observada: 201 Created. Pruebas aprobadas: 1/1.



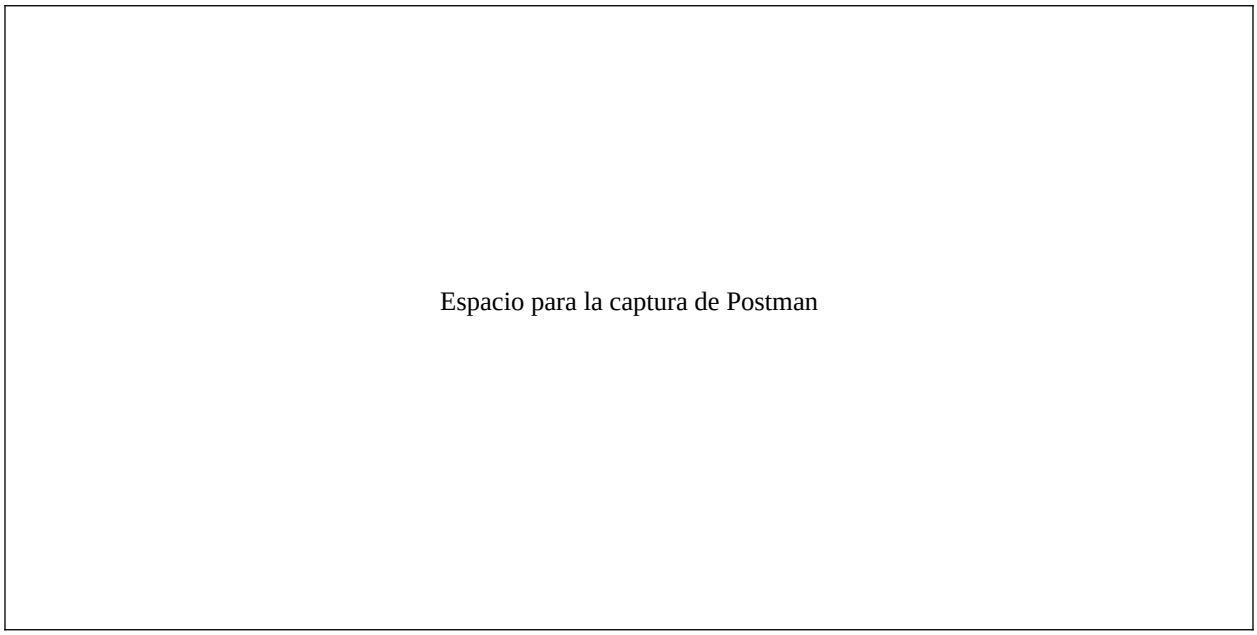
Se creó una solicitud con una zona distinta, terminada en -sin-conductores. El servidor devolvió el registro y estado PENDING_MATCH. La creación de la solicitud es válida aunque todavía no existan conductores compatibles; ese resultado se verifica en el siguiente caso.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 14. Matching sin conductores compatibles

POST /api/v1/trip-requests/{noMatchId}/match-proposals

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



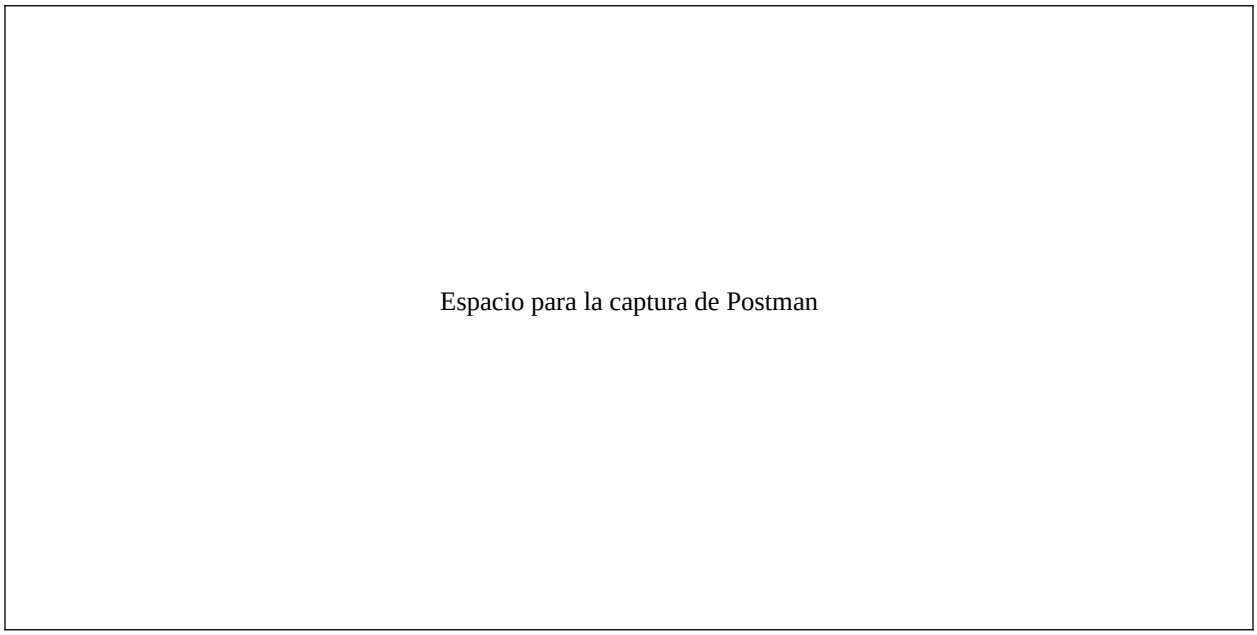
Se ejecutó el matching para la solicitud de la zona sin conductores. La respuesta fue una lista vacía, []. La consulta se procesó correctamente, pero no encontró candidatos para generar propuestas.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 15. Solicitud de cuatro asientos

POST /api/v1/trip-requests

Respuesta observada: 201 Created. Pruebas aprobadas: 1/1.



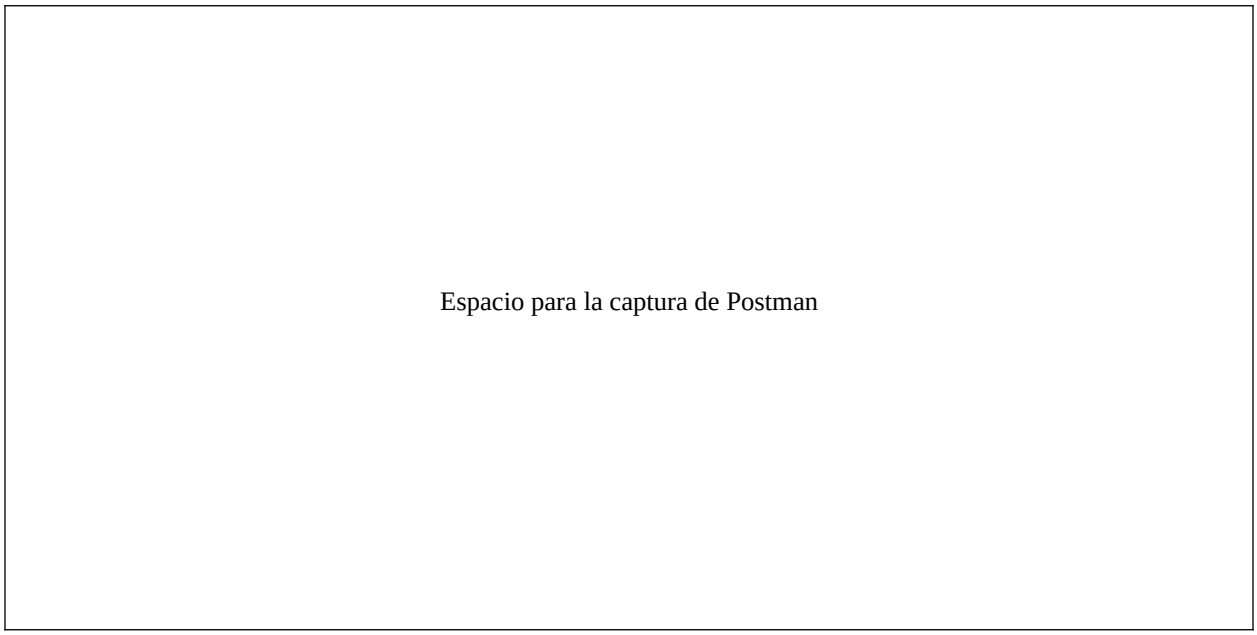
Se registró una solicitud en la zona original con seatsRequested igual a cuatro. El servidor devolvió un UUID, los cuatro asientos solicitados y el estado PENDING_MATCH. Este registro prepara la comprobación de capacidad del siguiente caso.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.

Figura 16. Matching con cupos insuficientes

POST /api/v1/trip-requests/{seatsId}/match-proposals

Respuesta observada: 200 OK. Pruebas aprobadas: 2/2.



Se generó el matching para la solicitud de cuatro asientos. La respuesta fue [], porque la disponibilidad registrada en esa zona ofrece tres asientos. El resultado confirma que ese conductor no se propone para una solicitud que supera su capacidad disponible.

Fuente: captura de Postman proporcionada por el autor, 1 de noviembre de 2026.